



第5章 访问数据库

- JDBC介绍
- JDBC API-JDBC主要接口
- 使用JDBC技术访问数据库
- 综合实例



学习目标

- 熟练掌握MySQL数据库的安装和使用
- 熟练掌握使用JDBC技术访问数据库的技术
- 熟练使用JDBC技术在动态网站中实现数据的增删改查等功能，并根据实际应用设计合适的SQL语句实现对数据库的操作



第5章 访问数据库

- JDBC介绍
- JDBC API-JDBC主要接口
- 使用JDBC技术访问数据库
- 综合实例



JDBC介绍

- JDBC (Java Database Connectivity) : Java 数据库连接, 是Java程序与数据库系统通信的标准API。
- 由一组用Java语言编写的类和接口组成
- JDBC可以做三件事
 - 与数据库建立连接
 - 发送SQL语句
 - 处理结果



JDBC总体结构

JDBC API

面向应用；提供访问关系型数据库的标准API；
java.sql及javax.sql包。

JDBC Driver API

Java Application

JDBC Manager

Driver A

JDBC-ODBC
Bridge Driver

ODBC and DB
Drivers



第5章 访问数据库

- JDBC介绍
- JDBC API-JDBC主要接口
- 使用JDBC技术访问数据库
- 综合实例



JDBC的主要接口

- java.sql.DriverManager
- java.sql.Connection
- java.sql.Statement
- java.sql.PreparedStatement
- java.sql.ResultSet



JDBC的主要接口

- `java.sql.DriverManager`
- `java.sql.Connection`
- `java.sql.Statement`
- `java.sql.PreparedStatement`
- `java.sql.ResultSet`

`DriverManager` 类是 `java.sql` 包中用于数据库驱动程序管理的类，作用于用户和驱动程序之间。用于建立应用程序到数据库的连接。



JDBC的主要接口

- java.sql.DriverManager
- java.sql.Connection
- java.sql.Statement
- java.sql.PreparedStatement
- java.sql.ResultSet

Connection是用来表示数据库连接的对象。在连接基础上可建立Statement语句对象或PreparedStatement预编译SQL语句对象。



JDBC的主要接口

- java.sql.DriverManager
- java.sql.Connection
- java.sql.Statement
- java.sql.PreparedStatement
- java.sql.ResultSet

Statement用于在已经建立的连接的基础上向数据库发送SQL语句的对象。包括了对数据库进行查询和更新等操作的方法。



JDBC的主要接口

- java.sql.DriverManager
- java.sql.Connection
- java.sql.Statement
- java.sql.PreparedStatement
- java.sql.ResultSet

PreparedStatement在已经建立的连接的基础上向数据库发送预编译的SQL语句，以?表示参数。其中除对数据库进行查询和更新等操作方法外，还提供了设置参数的方法
setXxx(int, Xxx)。



JDBC的主要接口

- java.sql.DriverManager
- java.sql.Connection
- java.sql.Statement
- java.sql.PreparedStatement
- java.sql.ResultSet

ResultSet以逻辑表格形式封装了执行数据库**查询**操作的结果集，包括游标基本控制方法next()及获取当前行某列数据的方法getXxx(int|String)。若结果集可滚动、可修改，还有其它方法。



第5章 访问数据库

- JDBC介绍
- JDBC API-JDBC主要接口
- 使用JDBC技术访问数据库
- 综合实例



使用JDBC技术访问数据库

- 准备工作：获取相应数据库的jdbc驱动程序，并将它们保存在指定的位置，然后重新启动tomcat服务；
- 利用JDBC访问数据库的具体步骤
 - 装载驱动程序；
 - 定义所要连接数据库的地址；
 - 建立与数据库的连接；
 - 建立语句对象；
 - 声明并执行SQL语句；
 - 处理返回的结果；
 - 关闭对象；



首先导入java.sql包

- 首先应该导入java.sql包。语句为：
`<%@page import="java.sql.*" %>`



编程步骤1——装载驱动程序

- 利用Class.forName()方法加载某一个数据库的驱动程序。
- 语法: `Class.forName("JDBC驱动程序类");`
- 方法:
 - 使用JDBC/ODBC桥驱动程序:
`Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");`
 - 使用MySQL的驱动程序:
 - 5. x 版本: `Class.forName("com.mysql.jdbc.Driver");`
 - 6. x 版本后: `Class.forName("com.mysql.cj.jdbc.Driver");`
 - 使用SQLServer的驱动程序:
`Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver");`



编程步骤1——装载驱动程序

- 方法：
 - 使用oracle的驱动程序：
`Class.forName("oracle.jdbc.driver.OracleDriver");`
 - 使用DB2的驱动程序：
`Class.forName("com.ibm.db2.jdbc.app.DB2Driver");`
- 经过以上其中之一语句的声明后，驱动程序就会自动地加载。
- 注意：如果classpath路径中没有找到所需的驱动程序则会抛出 **ClassNotFoundException**异常。



编程步骤2——定义连接数据库地址

- 数据库地址URL由3部分组成，各个部分用冒号间隔格式如下：**jdbc:<子协议>:<子名称>**
 - JDBC URL的3部分可分解如下：
 - jdbc: JDBC中的协议就是jdbc。
 - <子协议>: 数据库驱动程序名或数据库连接机制的名称。子协议的典型示例为oracle，它用于连接oracle数据库。
 - <子名称>: 一种标记数据库的方法。子名称根据子协议的不同而不同，使用子名称的目的是为定位数据库
 - 例如 jdbc:oracle:thin:@[ip]:[port]:[sid]
- 不同数据库连接的地址不同，这点需要特别注意，对一些**常见的数据库连接地址**，要求大家能够掌握。



编程步骤2——定义连接数据库地址

- 加载数据库驱动后，要对数据库建立连接，首先定义要连接的数据库地址，如：

```
String ODBCURL= " jdbc:odbc:dbName";
```

```
String mysqlURL=
```

```
"jdbc:mysql://host:port/dbName ";//因版本而异
```

```
String sqlserverURL= "
```

```
jdbc:microsoft:sqlserver://host:1433;DatabaseN  
ame=dbName ";
```

```
String oracleURL=
```

```
"jdbc:oracle:thin:@host:port:dbName ";
```

编程步骤3——建立与数据库的连接

- 语法: Connection
`con=DriverManager.getConnection(url,"登录数据库用户名","登录密码");`
- 例: 以mySQL连接数据库book为例, 用户名为root, 密码为root
 - `String driverStr="com.mysql.cj.jdbc.Driver";`////6.x版本
 - `String connStr="jdbc:mysql://localhost:3306/book";`
 - `Class.forName(driverStr);`
 - `Connection conn = DriverManager.getConnection(connStr, "root", "root");`



编程步骤4——建立语句对象

- 利用 Connection接口的 `createStatement()` 方法创建语句对象。
- 例： `Statement stmt=con.createStatement();`
- 语句对象用于执行SQL语句。该对象负责将SQL语句传递给数据库管理系统执行。
- 注意：语句对象的不同创建方法，可以导致不同的结果集（是否可滚动，是否可修改等）
 - 扩展阅读： [ResultSet用法集锦](#)



编程步骤5——声明并执行SQL语句

- 声明SQL语句，并将该语句通过Statement对象提交给服务器进行执行。
 - String sql= " SELECT * FROM book ";
 - //上面的语句用于从book表中查询所有记录。
 - ResultSet resultSet=stmt.executeQuery(sql);
 - //该语句将sql命令提交给数据库服务器进行执行，并将执行结果存储在ResultSet对象中进行执行。



编程步骤5——声明并执行SQL语句

- Statement接口主要有如下三个方法：

1. `public ResultSet executeQuery (String sql)`
`throws Exception`

以sql语句为参数，执行sql查询语句，返回一个ResultSet结果集（一个二维表）。

- 例如：

- `Statement stmt=con.createStatement();`
- `String sql= " SELECT * FROM book ";`
- `ResultSet rs=stmt.executeQuery(sql);`

编程步骤5——声明并执行SQL语句

- 2. `int executeUpdate(String sql)`
- 以sql语句为参数，执行sql更新语句及DDL语句，如添加、删除或修改操作，返回受影响记录的条数。
- 例如：
 - `Statement stmt=con.createStatement();`
 - `String sql= "delete FROM book where bookid= 12335";`
 - `int I =stmt.executeUpdate(sql);`
 - `System.out.println(i);`

编程步骤5——声明并执行SQL语句

- 3. **boolean execute**(String sql)
- 用于执行不知道类型的SQL语句，既可以执行查询语句，也可以执行更新语句。事先无法知道该SQL语句的具体类型及执行的返回值。
 - 当SQL语句执行后**返回ResultSet结果集时**，**返回true**；需通过Statement→getResultSet()方法得到返回的结果集
 - 当SQL语句执行后**没有返回结果集时**，**返回false**；需使用getUpdateCount()返回受影响的记录条数



编程步骤6——对结果集进行处理

- **ResultSet对象是执行数据库查询操作的结果集。**
 - 游标操作方法：
 - 基本操作方法 **next()**
 - 若为可滚动结果集，还有相关浏览数据的方法(first(), last(), previous(), absolute(int), ...) 方法
 - 结果集可修改时相应方法(updateString(col,val), ...)
- 支持多个结果集
 - 注意：多个结果集注意的问题
 - 在执行stmt.executeQuery的时候，会自动关闭上一次stmt.executeQuery的结果集。



ResultSet 有如下几个方法：

1. boolean next()

- 用于将 ResultSet 定位到下一行。
- ResultSet 最初是定位在第 0 行上的，此时应该先调用 next() 方法将其定位到第 1 行上，然后才可输出。
- 当 ResultSet 已经到了最后一行时，再调用 next() 方法，则返回 false。



2. getXxx()

- 当 ResultSet 已经定位在某一行上时, 使用 getXxx () 方法得到这一行上单个字段的值。
- 针对不同的字段类型, 调用不同的 getXxx () 方法 (getString(col), getDate(col), getInt(col), ...)
- 例:
 - 如果表中的第一个字段, 字段名为 “name”, 是文本型的; 则可以使用 getString(1) 或 getString(“name”) 来得到它的值。
 - 如果表中的第三个字段, 字段名为 “score”, 是数值型的; 则可以使用 getFloat(3) 或 getFloat(“score”) 来得到它的值。



2. getXXX()

- 显示结果集中所有记录的前两列:

```
while(rs.next()){  
    name =rs.getString(1);  
    phone =rs.getString(2);  
    System.out.println(name+","+phone);  
}
```



编程步骤7——关闭打开的资源

- `rs.close();`
//关闭结果集
- `stmt.close();`
//关闭语句对象
- `con.close();`
//关闭连接
- 注意：资源关闭的顺序和打开的顺序恰恰相反。



编程步骤8——异常处理

```
try
{
    ...
}
catch(SQLException ex)
{
    ...
}
```

例子

```
import java.sql.*
```

```
...
```

```
try{
```

```
Class.forName("com.mysql.jdbc.Driver");
```

```
String url="jdbc:mysql://localhost:3306/book";
```

```
Connection con=DriverManager.getConnection(url,"root","root");
```

```
Statement statement =con.createStatement();
```

```
String sql = "select bookname, author from bookinfo";
```

```
ResultSet rs= statement.executeQuery(sql);
```

```
while(rs.next()){
```

```
    bookname =rs.getString(1);
```

```
    author =rs.getString(2);
```

```
    System.out.println(bookname+","+author);
```

```
}
```

```
rs.close();
```

```
statement.close();
```

```
con.close();
```

```
}catch(SQLException e){...}
```



第一步



第二步



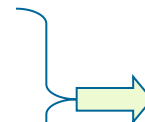
第三步



第四步



第五步



第六步



第七步



第5章 访问数据库

- JDBC介绍
- JDBC API-JDBC主要接口
- 使用JDBC技术访问数据库
- 综合实例

综合实例——图书信息管理系统

- 实现图书信息管理系统—图书信息的增删改查功能。
- 代码参阅ch06/文件夹，注意：
 - 驱动程序Jar包及驱动程序完整类名
 - 数据库连接URL
 - form及table标签的使用，表单数据的验证
 - 隐藏域的使用
 - 单击超链接时数据的传送
 - 请求的中文编码支持
 - 预编译语句对象的使用
 - DriverManager、Connection、Statement、PreparedStatement、ResultSet常用方法
 - 自动刷新的实现



提高

- 本章案例中，关于数据库的操作有大量重复代码，想一想如何改进？



小结

- 熟练掌握JDBC常用API
 - `java.sql.DriverManager`
 - `java.sql.Connection`
 - `java.sql.Statement`
 - `java.sql.PreparedStatement`
 - `java.sql.ResultSet`
- 掌握利用JDBC访问数据库的步骤，能利用JDBC技术实现对动态网站数据的增删改查。



作业

用户名:

密码:

确认密码:

E_mail:

请选择你的职业: ☒ 工人 ☐ 农民 ☐ 军人 ☐ 学生

请选择你的爱好: ☐ 音乐 ☐ 旅游 ☐ 读书

请输入你的留言:

- 编写上面的HTML页面register.htm
- 设计并写出相应的表结构
- 使用JDBC编写JSP页面register.jsp,将以上页面的数据存入数据库中